

Cryptographie

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction et principes fondamentaux

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Les objectifs de la cryptographie

La cryptographie moderne vise à garantir, en présence d'un adversaire potentiel, quatre propriétés fondamentales :

- **Confidentialité** : seules les parties autorisées peuvent accéder au contenu d'une information.
- **Intégrité** : toute modification non autorisée d'une information est détectable.
- **Authenticité** : l'origine d'une information (identité de l'émetteur) est vérifiable.
- **Non-répudiation** : l'émetteur d'un message ne peut pas nier ultérieurement en être l'auteur (propriété apportée notamment par la signature numérique).

2. Le principe de Kerckhoffs

Formulé par Auguste Kerckhoffs en 1883 : **la sécurité d'un système cryptographique ne doit reposer que sur le secret de la clé, jamais sur le secret de l'algorithme**. Un algorithme dont la sécurité dépend de rester confidentiel (« security through obscurity ») est fragile : il suffit qu'il soit divulgué ou rétro-ingénié une seule fois pour effondrer la sécurité de tous les systèmes qui l'utilisent. À l'inverse, un algorithme public, largement étudié par la communauté scientifique (comme AES, RSA), gagne en confiance précisément parce qu'il résiste à un examen ouvert et prolongé — lien direct avec le module *Cryptanalyse*.

3. Confusion et diffusion (Claude Shannon, 1949)

Deux propriétés structurantes de tout chiffrement robuste :

- **Confusion** : rendre la relation entre la clé et le texte chiffré la plus complexe possible, pour qu'une analyse statistique du chiffré ne révèle rien sur la clé.
- **Diffusion** : répartir l'influence de chaque bit du texte clair (et de la clé) sur l'ensemble du texte chiffré, pour qu'une modification d'un seul bit du clair change en moyenne la moitié des bits du chiffré (effet avalanche).

Ces deux notions justifient directement la structure des chiffrements par blocs modernes (tours de substitution et de permutation dans AES, chapitre 2).

4. Cryptographie symétrique vs asymétrique : vue d'ensemble

	Symétrique	Asymétrique
Clés	une seule clé, partagée entre émetteur et récepteur	une paire de clés (publique/privée)
Vitesse	rapide	beaucoup plus lente
Problème principal	distribution sécurisée de la clé partagée	vérification de l'authenticité de la clé publique
Exemples	AES, ChaCha20	RSA, Diffie-Hellman, ECC

En pratique, les systèmes réels combinent les deux : la cryptographie asymétrique établit ou échange une clé de session, ensuite utilisée pour un chiffrement symétrique rapide du volume de données (principe du **chiffrement hybride**, à l'œuvre notamment dans TLS, détaillé au chapitre 6).

5. Notion de sécurité prouvable et de sécurité calculatoire

La sécurité d'un système cryptographique moderne repose généralement sur une hypothèse calculatoire : un problème mathématique réputé difficile à résoudre en un temps raisonnable avec les moyens de calcul actuels et prévisibles (factorisation pour RSA, logarithme discret pour Diffie-Hellman/ECC). Il ne s'agit pas d'une preuve d'impossibilité absolue (à l'exception du **masque jetable** — *one-time pad* — seul système prouvé théoriquement incassable, mais impraticable car il exige une clé aussi longue que le message, utilisée une seule fois).

6. Vocabulaire de base

Terme	Définition
Texte clair (plaintext)	message original, non protégé
Texte chiffré (ciphertext)	résultat du chiffrement
Clé	paramètre secret contrôlant le chiffrement/déchiffrement
Algorithme (chiffre, cipher)	procédure de transformation, publique (Kerckhoffs)
Cryptosystème	ensemble algorithme + gestion des clés + protocole d'usage

À retenir

- Confidentialité, intégrité, authenticité, non-répudiation : quatre objectifs distincts, pas toujours tous nécessaires simultanément selon le contexte.
- Le principe de Kerckhoffs interdit de fonder la sécurité sur le secret de l'algorithme.
- La cryptographie moderne combine en pratique symétrique (rapide, volume) et asymétrique (échange de clé, authentification) dans des schémas hybrides.

Chapitre 2 — Cryptographie symétrique : chiffrement par bloc et par flux

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Chiffrement par flux (stream cipher)

Combine le texte clair avec une suite pseudo-aléatoire (le « keystream ») générée à partir de la clé, généralement par XOR bit à bit :

```
chiffré[i] = clair[i] XOR keystream[i]
```

Exemple moderne largement utilisé : **ChaCha20**. Un chiffrement par flux exige impérativement de **ne jamais réutiliser le même couple (clé, nonce)** : réutiliser le même keystream sur deux messages différents permet de retrouver le XOR des deux messages clairs, une fuite d'information souvent exploitable (rappel du chapitre Cryptanalyse).

2. Chiffrement par bloc (block cipher)

Chiffre des blocs de taille fixe (128 bits pour AES) en appliquant plusieurs « tours » de transformations combinant substitution et permutation.

AES (Advanced Encryption Standard)

Standardisé par le NIST en 2001 (algorithme Rijndael), AES opère sur des blocs de 128 bits avec des clés de 128, 192 ou 256 bits, en 10, 12 ou 14 tours respectivement. Chaque tour comprend classiquement :

1. **SubBytes** : substitution non linéaire octet par octet (boîte-S), assure la confusion.
2. **ShiftRows** : décalage cyclique des lignes de l'état, contribue à la diffusion.
3. **MixColumns** : combinaison linéaire des colonnes (absente au dernier tour), renforce la diffusion.
4. **AddRoundKey** : XOR avec une sous-clé dérivée de la clé principale pour ce tour.

3. Modes opératoires : pourquoi ils sont indispensables

Un chiffrement par bloc ne traite que des blocs de taille fixe : un **mode opératoire** définit comment enchaîner le chiffrement de plusieurs blocs pour traiter un message de longueur arbitraire.

Mode	Principe	Points clés
ECB (Electronic Codebook)	chaque bloc chiffré indépendamment	à proscrire : des blocs clairs identiques donnent des blocs chiffrés identiques (motifs visibles, cf. TP3 du module Cryptanalyse)
CBC (Cipher Block Chaining)	chaque bloc clair est XORé avec le bloc chiffré précédent avant chiffrement ; nécessite un IV aléatoire	vulnérable aux attaques de padding oracle si le padding n'est pas vérifié de façon sûre
CTR (Counter)	transforme un chiffrement par bloc en chiffrement par flux, en chiffrant un compteur incrémental	parallélisable, mais un nonce réutilisé est aussi dangereux qu'en chiffrement par flux
GCM (Galois/Counter Mode)	mode CTR combiné à une authentification intégrée (AEAD)	recommandé par défaut : fournit confidentialité ET intégrité/authenticité en une seule primitive

4. AEAD : chiffrement authentifié

Un mode **AEAD** (Authenticated Encryption with Associated Data), comme AES-GCM ou ChaCha20-Poly1305, combine chiffrement et code d'authentification en une seule opération, produisant un **tag d'authentification** vérifié au déchiffrement. Si le texte chiffré a été modifié (accidentellement ou par un attaquant), le déchiffrement échoue explicitement plutôt que de produire silencieusement un texte clair corrompu ou manipulé.

Recommandation pratique actuelle : privilégier systématiquement un mode AEAD (AES-GCM, ChaCha20-Poly1305) plutôt que de composer soi-même un mode CBC avec un HMAC séparé — une composition manuelle est une source fréquente d'erreurs d'implémentation.

5. Gestion de la clé et du vecteur d'initialisation (IV)/nonce

- La **clé** doit être générée par un générateur aléatoire cryptographiquement sûr (CSPRNG) et jamais codée en dur dans le code source.
- L'**IV/nonce** doit être unique pour chaque chiffrement avec une même clé (aléatoire pour CBC, souvent un compteur pour CTR/GCM) — sa réutilisation est l'une des erreurs d'implémentation les plus dommageables en pratique.

6. Dérivation de clé à partir d'un mot de passe

Un mot de passe humain n'a pas l'entropie suffisante pour servir directement de clé de chiffrement : on utilise une **fonction de dérivation de clé (KDF)** dédiée (PBKDF2, scrypt, Argon2), volontairement coûteuse en calcul, pour ralentir les attaques par force brute (rappel du chapitre Cryptanalyse sur le hachage de mots de passe).

À retenir

- AES est le standard de chiffrement par bloc de référence ; sa sécurité dépend fortement du mode opératoire choisi.
- ECB est à proscrire ; les modes AEAD (GCM) sont recommandés par défaut car ils apportent confidentialité et intégrité simultanément.
- La réutilisation d'un nonce/IV avec la même clé est l'une des erreurs d'implémentation les plus graves en cryptographie symétrique.

Chapitre 3 — Fonctions de hachage et authentification de message (MAC/HMAC)

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Fonctions de hachage cryptographiques

Une fonction de hachage H transforme une entrée de taille arbitraire en une sortie de taille fixe (l'empreinte ou *digest*), avec trois propriétés attendues (détaillées côté attaque au chapitre 4 du module *Cryptanalyse*) :

- résistance à la préimage,
- résistance à la seconde préimage,
- résistance aux collisions.

Panorama des fonctions usuelles

Fonction	Taille de sortie	Statut
MD5	128 bits	cassée (collisions pratiques), à ne plus utiliser pour un usage sécuritaire
SHA-1	160 bits	cassée (collision démontrée en 2017), dépréciée
SHA-2 (SHA-256, SHA-384, SHA-512)	256 à 512 bits	recommandée, aucune attaque pratique connue
SHA-3 (Keccak)	224 à 512 bits	recommandée, construction interne différente de SHA-2 (fonction éponge), diversifie le risque en cas de faille future sur la famille SHA-2

2. Usages des fonctions de hachage

- **Vérification d'intégrité** : comparer l'empreinte d'un fichier téléchargé à une valeur de référence publiée.
- **Structures de données** : arbres de Merkle (utilisés en Blockchain, module suivant), tables de hachage.
- **Brique de construction** : dérivation de clé, génération de nombres pseudo-aléatoires, signatures numériques (chapitre 5), preuve de travail.
- **Hachage de mots de passe** : nécessite des fonctions dédiées et volontairement lentes (bcrypt, scrypt, Argon2), jamais une fonction générique seule (cf. module Cryptanalyse, chapitre 4).

3. Code d'authentification de message (MAC)

Un MAC garantit à la fois **l'intégrité** d'un message et **l'authenticité** de son origine, pour deux parties partageant une clé secrète : $MAC = F_k(message)$, où seule une partie possédant la clé k peut produire ou vérifier un MAC valide.

À la différence d'une signature numérique (chapitre 5), un MAC ne fournit pas la non-répudiation : toute partie capable de vérifier le MAC est également capable d'en produire un (elle possède la même clé).

4. HMAC (Hash-based MAC)

Construction générique permettant de bâtir un MAC à partir de n'importe quelle fonction de hachage :

$$\text{HMAC}(k, m) = H((k' \text{ XOR } \text{opad}) || H((k' \text{ XOR } \text{ipad}) || m))$$

où k' est la clé complétée à la taille de bloc de H , et ipad/opad des constantes fixes. Cette double application de H , avec la clé combinée de part et d'autre, neutralise spécifiquement l'attaque par extension de longueur qui rend dangereuse une simple concaténation $H(\text{clé} || \text{message})$ (vue au chapitre 4 du module *Cryptanalyse*).

HMAC-SHA256 est aujourd'hui l'un des mécanismes de MAC les plus utilisés (authentification d'API, jetons JWT signés en HS256, intégrité de sessions).

5. MAC intégré : GMAC et Poly1305

Les modes AEAD vus au chapitre précédent (AES-GCM, ChaCha20-Poly1305) intègrent directement un mécanisme de MAC (GMAC pour GCM, Poly1305 pour ChaCha20) dans l'opération de chiffrement elle-même, évitant d'avoir à composer manuellement chiffrement et authentification — une composition manuelle mal ordonnée (ex. authentifier avant de chiffrer plutôt que l'inverse) est une source d'erreurs classique.

6. Ordre chiffrement/authentification (Encrypt-then-MAC)

Lorsqu'un schéma AEAD intégré n'est pas utilisé et qu'il faut composer soi-même chiffrement et MAC, la construction recommandée est **Encrypt-then-MAC** (chiffrer d'abord, puis authentifier le texte chiffré) plutôt que MAC-then-Encrypt ou Encrypt-and-MAC, car elle permet de rejeter un texte chiffré invalide avant même de tenter de le déchiffrer, limitant la surface d'attaques par oracle (padding oracle notamment).

| À retenir

- Un MAC garantit intégrité et authenticité pour des parties partageant une clé secrète, sans non-répudiation.
- HMAC neutralise spécifiquement l'attaque par extension de longueur inhérente aux fonctions de hachage basées sur Merkle-Damgård.
- Privilégier un mode AEAD intégré (GCM, Poly1305) plutôt que de composer manuellement chiffrement et MAC ; si nécessaire, appliquer le principe Encrypt-then-MAC.

Chapitre 4 — Cryptographie asymétrique : RSA, Diffie-Hellman, ECC

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe général

Chaque partie dispose d'une paire de clés mathématiquement liées : une **clé publique**, diffusable librement, et une **clé privée**, gardée strictement secrète. Deux usages principaux : chiffrer avec la clé publique du destinataire (lui seul peut déchiffrer avec sa clé privée), ou signer avec sa propre clé privée (tout le monde peut vérifier avec la clé publique correspondante — chapitre 5).

2. Rappels d'arithmétique modulaire nécessaires

- **Congruence modulo n** : $a \equiv b \pmod{n}$ si a et b ont le même reste dans la division par n .
- **Exponentiation modulaire** : $a^e \pmod{n}$, calculable efficacement par l'algorithme d'exponentiation rapide (« square and multiply »), sans jamais calculer a^e en entier.
- **Petit théorème de Fermat / théorème d'Euler** : fondement de la construction de RSA.
- **Inverse modulaire** : d tel que $e \times d \equiv 1 \pmod{\varphi(n)}$, calculable par l'algorithme d'Euclide étendu — c'est ainsi que la clé privée RSA est dérivée de la clé publique et des facteurs premiers.

3. RSA : construction

1. Choisir deux grands nombres premiers distincts p et q ; calculer $n = p \times q$.
2. Calculer $\varphi(n) = (p-1)(q-1)$ (indicatrice d'Euler).
3. Choisir un exposant public e premier avec $\varphi(n)$ (souvent 65537).
4. Calculer l'exposant privé d , inverse modulaire de e modulo $\varphi(n)$.
5. Clé publique : (n, e) . Clé privée : (n, d) .

Chiffrement : $c = m^e \bmod n$. **Déchiffrement** : $m = c^d \bmod n$.

La sécurité repose sur la difficulté de retrouver d (ou de factoriser n) sans connaître p et q (rappel : chapitre 5 du module *Cryptanalyse*).

Padding : une nécessité, pas une option

RSA « brut » (sans padding) est déterministe et vulnérable à plusieurs attaques structurelles (cf. module *Cryptanalyse*). Les standards actuels imposent un schéma de padding : **OAEP** pour le chiffrement, **PSS** pour la signature (chapitre 5). Utiliser RSA sans padding correct est une erreur d'implémentation grave, quelle que soit la taille de clé.

4. Échange de clé Diffie-Hellman

Permet à deux parties d'établir un secret partagé sur un canal public, sans jamais transmettre ce secret directement :

1. Paramètres publics : un nombre premier p et un générateur g .
2. Alice choisit un secret a , calcule et envoie $A = g^a \bmod p$.
3. Bob choisit un secret b , calcule et envoie $B = g^b \bmod p$.
4. Alice calcule $B^a \bmod p$, Bob calcule $A^b \bmod p$: les deux obtiennent la même valeur $g^{(ab)} \bmod p$, le secret partagé.

Un observateur du canal voit p , g , A et B , mais retrouver a ou b (ou directement $g^{(ab)}$) requiert de résoudre le problème du logarithme discret, réputé difficile pour des paramètres suffisamment grands.

Limite importante : Diffie-Hellman « pur » ne fournit aucune authentification — il est vulnérable à une attaque de l'homme du milieu (*man-in-the-middle*) si les parties ne vérifient pas par ailleurs l'identité de leur interlocuteur (rôle des certificats et signatures, chapitre 5).

5. Cryptographie sur courbes elliptiques (ECC)

Remplace le groupe multiplicatif modulo p par le groupe des points d'une courbe elliptique définie sur un corps fini. Le problème équivalent au logarithme discret (ECDLP) y est considéré plus difficile à taille de clé égale, permettant des clés bien plus courtes pour un niveau de sécurité équivalent (rappel chapitre 5 du module *Cryptanalyse* : ECC 256 bits $\approx$ RSA 3072 bits). Courbes largement utilisées aujourd'hui : P-256 (NIST), Curve25519 (utilisée par X25519 pour l'échange de clé et Ed25519 pour la signature, chapitre 5).

6. Chiffrement hybride

En pratique, la cryptographie asymétrique n'est presque jamais utilisée pour chiffrer directement de gros volumes de données (trop lente) : elle sert à établir ou transmettre une **clé de session**, ensuite utilisée avec un chiffrement symétrique rapide (AES-GCM). C'est le principe exact mis en œuvre par TLS (chapitre 6).

À retenir

- RSA repose sur la difficulté de factorisation ; Diffie-Hellman et ECC reposent sur la difficulté du logarithme discret (classique ou sur courbe elliptique).
- Le padding (OAEP/PSS) n'est pas optionnel : RSA brut est vulnérable à des attaques structurelles connues.
- Diffie-Hellman pur n'authentifie pas les parties ; il doit être combiné à un mécanisme d'authentification pour résister à une attaque de l'homme du milieu.

Chapitre 5 — Signatures numériques et infrastructures à clés publiques (PKI)

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe de la signature numérique

La signature numérique fournit **authenticité**, **intégrité** et **non-répudiation** : le signataire utilise sa clé privée pour produire une signature liée à un message, que quiconque peut vérifier avec la clé publique correspondante, sans pouvoir produire lui-même une signature valide.

Schéma général

1. Calculer l'empreinte du message : $h = H(\text{message})$ (fonction de hachage, chapitre 3).
2. Signer l'empreinte avec la clé privée : $\text{signature} = \text{Sign}(\text{clé privée}, h)$.
3. Vérification : recalculer $H(\text{message})$ et vérifier la signature avec la clé publique.

Signer l'empreinte plutôt que le message entier permet de traiter des messages de taille arbitraire avec des opérations cryptographiques de taille fixe, et rend toute modification du message détectable (l'empreinte changerait).

2. RSA-PSS

Schéma de signature basé sur RSA avec un padding probabiliste (PSS — Probabilistic Signature Scheme), aujourd'hui recommandé pour la signature (par opposition à un ancien schéma déterministe PKCS#1 v1.5, encore présent mais moins recommandé pour de nouveaux déploiements).

3. ECDSA et Ed25519

- **ECDSA** : signature sur courbe elliptique, largement déployée (certificats TLS, Bitcoin). Point de vigilance : la sécurité d'ECDSA dépend crucialement d'un nombre aléatoire (« nonce ») unique et non prévisible à chaque signature — un nonce réutilisé ou prévisible permet de retrouver directement la clé privée (illustré par des compromissions réelles, notamment sur certaines consoles de jeu et portefeuilles Bitcoin mal implémentés).
- **Ed25519** : schéma de signature moderne basé sur la courbe Curve25519, déterministe (ne dépend pas d'un aléa externe à chaque signature, éliminant le risque de nonce faible), rapide et largement recommandé pour les nouveaux déploiements (SSH, protocoles modernes).

4. Certificats numériques

Un certificat numérique lie une identité (personne, serveur, organisation) à une clé publique, la liaison étant elle-même garantie par la signature numérique d'une **autorité de certification (AC/CA)** de confiance. Format standard le plus répandu : **X.509**.

Contenu typique d'un certificat X.509

- Identité du sujet (ex. nom de domaine pour un certificat de serveur).
- Clé publique du sujet.
- Identité de l'autorité de certification émettrice.
- Période de validité.
- Signature numérique de l'AC sur l'ensemble de ces informations.

5. Infrastructure à clés publiques (PKI)

Ensemble des composants et procédures permettant de créer, distribuer, vérifier et révoquer des certificats :

- **Autorité de certification (AC/CA)** : émet et signe les certificats.
- **Autorité d'enregistrement (RA)** : vérifie l'identité du demandeur avant émission.
- **Chaîne de confiance** : un certificat intermédiaire est lui-même signé par une AC racine, dont la clé publique est préinstallée (« ancre de confiance ») dans les navigateurs et systèmes d'exploitation.
- **Révocation** : mécanismes de liste de révocation (CRL) ou de vérification en ligne (OCSP), permettant d'invalidier un certificat compromis avant sa date d'expiration normale.

Vérification d'un certificat côté client

1. Le certificat est-il signé par une AC de confiance (remontée de la chaîne jusqu'à une ancre de confiance) ?
2. Le certificat est-il dans sa période de validité ?
3. Le certificat correspond-il bien à l'identité attendue (ex. nom de domaine) ?
4. Le certificat a-t-il été révoqué ?

Une faille dans n'importe laquelle de ces vérifications (ex. accepter un certificat expiré, ou ne pas vérifier le nom de domaine) annule la protection apportée par toute la chaîne cryptographique — un point classique d'audit de sécurité applicative (module *Audit organisation et technique*, chapitre 5).

6. Modèles alternatifs

- **Web of Trust** (utilisé historiquement par PGP/GPG) : confiance distribuée entre pairs plutôt que hiérarchique, sans autorité centrale.
- **Certificate Transparency** : registres publics et vérifiables des certificats émis, permettant de détecter des émissions frauduleuses par une AC compromise ou négligente.

| À retenir

- La signature numérique apporte la non-répudiation, absente d'un simple MAC (chapitre 3).
- ECDSA exige un nonce aléatoire de haute qualité à chaque signature ; Ed25519 élimine ce risque par construction déterministe.
- Une PKI repose sur une chaîne de confiance hiérarchique ; sa robustesse dépend entièrement de la rigueur de vérification à chaque maillon (identité, validité, révocation).

Chapitre 6 — Protocoles appliqués : TLS et cryptographie post-quantique

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. TLS (Transport Layer Security) : rôle et positionnement

TLS sécurise les communications entre un client et un serveur (HTTPS, mais aussi d'autres protocoles applicatifs) en fournissant confidentialité, intégrité et authentification du serveur (et optionnellement du client). Il illustre concrètement l'intégration de toutes les primitives vues dans ce module.

2. La poignée de main TLS 1.3 (simplifiée)

1. **ClientHello** : le client propose les suites cryptographiques supportées et envoie des paramètres pour un échange de clé (ex. clé publique éphémère pour Diffie-Hellman sur courbe elliptique).
2. **ServerHello** : le serveur choisit une suite cryptographique, complète l'échange de clé et envoie son certificat.
3. **Vérification du certificat** : le client valide la chaîne de confiance jusqu'à une AC racine reconnue (chapitre 5).
4. **Dérivation des clés de session** : à partir du secret partagé issu de l'échange de clé, une fonction de dérivation de clé produit les clés symétriques utilisées pour le reste de la session.
5. **Communication chiffrée** : l'ensemble du trafic applicatif est protégé par un chiffrement symétrique authentifié (AES-GCM ou ChaCha20-Poly1305).

TLS 1.3 (2018) a simplifié la poignée de main par rapport à TLS 1.2, supprimé les suites cryptographiques jugées faibles (RC4, CBC sans protection adéquate, RSA pour l'échange de clé sans confidentialité persistante), et impose désormais un échange de clé Diffie-Hellman éphémère par défaut.

3. Confidentialité persistante (Perfect Forward Secrecy)

Propriété garantissant que la compromission ultérieure de la clé privée à long terme d'un serveur ne permet pas de déchiffrer rétroactivement des sessions passées interceptées et enregistrées. Obtenue en utilisant des clés d'échange **éphémères** (générées pour chaque session puis détruites), typiquement via Diffie-Hellman éphémère sur courbe elliptique (ECDHE). C'est une des raisons pour lesquelles TLS 1.3 impose ce mécanisme par défaut.

4. Suites cryptographiques et obsolescence

Une « suite cryptographique » combine un mécanisme d'échange de clé, un algorithme de signature, un chiffrement symétrique et une fonction de hachage. Les recommandations évoluent avec le temps à mesure que des faiblesses sont découvertes (rappel du module *Cryptanalyse*) : SSL 2.0/3.0, TLS 1.0/1.1, RC4, l'export de clés faibles (« EXPORT »), sont aujourd'hui déconseillés voire interdits par les référentiels de sécurité (ANSSI, PCI-DSS).

5. Erreurs d'implémentation TLS fréquentes (audit)

- Ne pas vérifier le nom de domaine du certificat.
- Accepter des certificats auto-signés ou expirés en production.
- Désactiver la vérification de certificat pendant le développement, oubli en production (vulnérabilité réelle et récurrente).
- Utiliser des suites cryptographiques obsolètes non désactivées côté serveur.

Ces points recourent directement la grille d'audit applicatif vue dans le module *Audit organisation et technique*.

6. La menace post-quantique et la migration en cours

L'algorithme de Shor (rappel : module *Cryptanalyse*, chapitre 5), exécuté sur un ordinateur quantique suffisamment puissant, casserait RSA, Diffie-Hellman et ECC classiques. Bien qu'un tel ordinateur ne soit pas disponible aujourd'hui à l'échelle nécessaire, deux considérations motivent une action dès maintenant :

- « **Harvest now, decrypt later** » : un adversaire peut enregistrer aujourd'hui du trafic chiffré intercepté, dans l'espoir de le déchiffrer plus tard une fois l'informatique quantique suffisamment mature — un risque réel pour des données à confidentialité longue durée.
- **Délai de migration** : le déploiement d'une nouvelle famille cryptographique à l'échelle d'Internet prend typiquement plusieurs années.

Le NIST a standardisé en 2024 des algorithmes post-quantiques (**ML-KEM**, anciennement Kyber, pour l'échange de clé ; **ML-DSA**, anciennement Dilithium, pour la signature), dont l'intégration progressive dans TLS et d'autres protocoles est en cours, souvent sous forme **hybride** (combinant un mécanisme classique et un mécanisme post-quantique, pour ne pas dépendre uniquement de la maturité encore récente de ces nouveaux algorithmes).

À retenir

- TLS combine, de façon opérationnelle, échange de clé asymétrique, authentification par certificat et chiffrement symétrique authentifié — la synthèse pratique de tout ce module.
- La confidentialité persistante (clés éphémères) protège les échanges passés même en cas de compromission future d'une clé long terme.
- La transition post-quantique est engagée par anticipation, notamment face au risque « harvest now, decrypt later » sur des données sensibles à long terme.